

LAPORAN PRAKTIKUM

Workshop Web Dasar

Persiapan Lab Environment

*(Implementasi Dasar Git dan GitHub untuk Manajemen
Repository)*



**DISUSUN OLEH:
ZULFAN SAHPUTRA
2025573010031
TEKNIK INFORMATIKA**

POLITEKNIK NEGERI LHOKSEUMAWE

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum ini dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan pada mata kuliah Workshop Web Dasar.

Dalam proses penyusunan laporan praktikum ini, penulis memperoleh banyak pengetahuan serta pengalaman baru, khususnya dalam memahami lingkungan kerja laboratorium serta penggunaan teknologi pendukung pengembangan web. Oleh karena itu, laporan ini diharapkan dapat memberikan gambaran mengenai proses praktikum yang telah dilakukan serta menjadi bahan pembelajaran bagi penulis maupun pembaca.

Penulis juga menyampaikan ucapan terima kasih kepada Bapak Muhammad Reza Zulman, S.ST., M.Sc. selaku dosen pengampu mata kuliah Workshop Web Dasar yang telah memberikan arahan, bimbingan, serta ilmu selama kegiatan perkuliahan dan praktikum berlangsung.

Penulis menyadari bahwa laporan praktikum ini masih memiliki kekurangan, baik dari segi penyusunan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa yang akan datang.

Akhir kata, penulis berharap laporan praktikum ini dapat memberikan manfaat serta menambah wawasan bagi para pembaca.

Tempat, Tanggal

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM	1
KATA PENGANTAR.....	2
DAFTAR ISI	3
BAB I PENDAHULUAN	5
1.1. Latar Belakang.....	5
1.2. Identifikasi Masalah	5
1.3. Rumusan Masalah	6
1.4. Tujuan Praktikum	6
BAB II LANDASAN TEORI	7
2.1. Konsep Version Control System (VCS)	7
2.2. Git.....	7
2.3. GitHub	8
2.4. Alur Kerja Git (Git Workflow)	8
2.5. Fungsi Command Dasar Git	9
BAB III METODOLOGI PRAKTIKUM.....	10
3.1. Alat dan Bahan	10
3.2. Prosedur Praktikum	10
3.2.1. Registrasi Akun GitHub	10
3.2.2. Membuat repository baru pada GitHub	12
3.2.3. Cloning repository GitHub ke local	13
3.2.4. Editing README.md dan sinkronisasi repository	15
BAB IV HASIL DAN PEMBAHASAN	16
4.1. Hasil Praktikum	16
4.2. Pembahasan	16
BAB V KESIMPULAN DAN SARAN	17
5.1. Kesimpulan.....	17
5.2. Saran.....	17

DAFTAR PUSTAKA.....	18
---------------------	----

BAB I

PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi informasi yang pesat telah mendorong perubahan dalam metode pengembangan perangkat lunak. Salah satu aspek penting dalam proses tersebut adalah pengelolaan perubahan kode agar setiap perubahan dapat tercatat dengan baik dan tidak menimbulkan konflik antar versi file. Tanpa sistem yang terstruktur, perubahan file berisiko hilang, tertimpa, atau sulit dilacak. Oleh karena itu, diperlukan suatu sistem yang mampu mencatat dan mengelola perubahan secara sistematis, yaitu *Version Control System (VCS)* seperti Git.

Git memungkinkan pengembang menyimpan riwayat perubahan, kembali ke versi sebelumnya, serta bekerja secara kolaboratif dengan lebih terorganisir. Dengan dukungan platform berbasis web seperti GitHub, pengelolaan *repository* dapat dilakukan secara online dan sinkron antara perangkat lokal dan *server*. Oleh karena itu, praktikum ini bertujuan untuk memahami implementasi dasar Git dan GitHub, mulai dari pembuatan *repository* hingga proses *clone*, *commit*, *push*, dan *pull* sebagai dasar pengelolaan proyek perangkat lunak.

1.2. Identifikasi Masalah

Berdasarkan latar belakang tersebut, dapat diidentifikasi beberapa permasalahan:

1. Banyak mahasiswa belum memahami konsep dasar version control.
2. Masih terdapat kebingungan dalam membedakan *repository* lokal dan *repository* remote.
3. Kurangnya pemahaman mengenai alur kerja Git seperti *staging*, *commit*, *push*, dan *pull*.

1.3. Rumusan Masalah

1. Apa yang dimaksud dengan *Git* dan *GitHub*?
2. Bagaimana proses pembuatan *repository* pada *GitHub*?
3. Bagaimana alur kerja *Git* dalam melakukan *staging*, *commit*, *push* dan *pull*?
4. Bagaimana proses sinkronisasi data antara local *repository* dan remote *repository*?

1.4. Tujuan Praktikum

1. Memahami konsep dasar sistem kontrol versi menggunakan *Git*.
2. Mengimplementasikan pembuatan *repository* pada *GitHub*.
3. Melakukan *cloning repository* ke perangkat lokal.
4. Mempraktikkan proses *staging*, *commit*, dan *push*.
5. Melakukan sinkronisasi data menggunakan *pull*.

BAB II

LANDASAN TEORI

2.1. Konsep Version Control System (VCS)

Version Control adalah sistem yang mencatat perubahan pada suatu file atau sekumpulan file dari waktu ke waktu sehingga Anda dapat memanggil kembali versi tertentu di kemudian hari. Secara umum, VCS terbagi menjadi dua jenis:

a. Centralized Version Control System (CVCS)

Sistem ini memiliki satu server pusat sebagai tempat penyimpanan utama.

b. Distributed Version Control System (DVCS)

Setiap pengguna memiliki salinan lengkap repository. Git termasuk dalam kategori ini.

Keunggulan DVCS adalah tidak bergantung sepenuhnya pada server pusat dan memungkinkan kerja offline.

2.2. Git

Git adalah sistem kontrol versi terdistribusi yang dikembangkan oleh Linus Torvalds pada tahun 2005 untuk mendukung pengembangan kernel Linux.

Git bekerja dengan konsep snapshot, bukan menyimpan perubahan baris per baris. Setiap *commit* menyimpan kondisi file pada waktu tertentu. Jika tidak ada perubahan pada file tertentu, Git hanya mereferensikan file sebelumnya tanpa menduplikasinya.

a. Konsep penting dalam Git:

b. *Repository*

c. Working Directory

d. *Staging Area*

e. *Commit*

f. *Remote Repository*

2.3. *GitHub*

GitHub adalah platform hosting *repository* berbasis Git yang menyediakan layanan kolaborasi pengembangan perangkat lunak secara daring. GitHub memungkinkan penyimpanan kode secara online, pelacakan perubahan, manajemen issue, serta integrasi dengan berbagai tools CI/CD.

GitHub bertindak sebagai *remote repository* yang memungkinkan sinkronisasi antara perangkat lokal dengan server.

2.4. *Alur Kerja Git (Git Workflow)*

Alur kerja dasar Git terdiri dari beberapa tahapan:

1. *Working Directory*
Tempat file diedit.
2. *Staging Area*
Tempat file dipersiapkan sebelum *commit*.
3. *Local Repository*
Tempat *commit* disimpan secara lokal.
4. *Remote Repository*
Penyimpanan di server (GitHub).

Proses ini menciptakan sistem pengelolaan versi yang terstruktur dan aman.

2.5. Fungsi Command Dasar Git

Dalam praktik Git, terdapat empat perintah dasar yang sering digunakan untuk mengelola *repository*, yaitu `add`, `commit`, `push`, dan `pull`. Fungsi masing-masing adalah sebagai berikut:

<i>Command</i>	<i>Fungsi</i>
<code>git add</code>	Menandai file atau perubahan agar siap dicatat dalam <i>commit</i>
<code>git commit</code>	Menyimpan <i>snapshot</i> dari file yang sudah di-add ke <i>repository</i> lokal
<code>git push</code>	Mengirim <i>commit</i> dari <i>repository</i> lokal ke <i>repository</i> remote (GitHub)
<code>git pull</code>	Mengambil dan menggabungkan perubahan terbaru dari <i>repository</i> remote

Alur kerja singkat perintah Git:

1. **Working Directory** → mengedit file
2. **Staging Area (`git add`)** → menandai file yang siap dicatat
3. **Local Repository (`git commit`)** → menyimpan *snapshot* secara lokal
4. **Remote Repository (`git push`)** → mengirim *commit* ke GitHub
5. **Sinkronisasi perubahan terbaru (`git pull`)** → mengambil update dari remote ke lokal

BAB III

METODOLOGI PRAKTIKUM

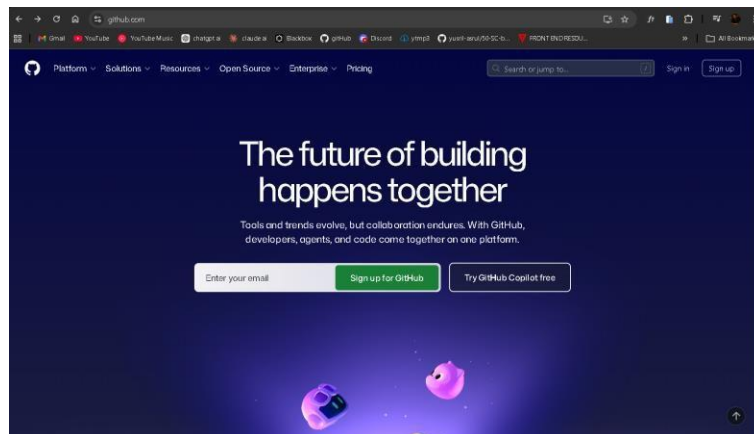
3.1. *Alat dan Bahan*

- a. Laptop
- b. Browser
- c. Git Bash
- d. Akun GitHub
- e. Visual Studio Code
- f. Jaringan internet

3.2. *Prosedur Praktikum*

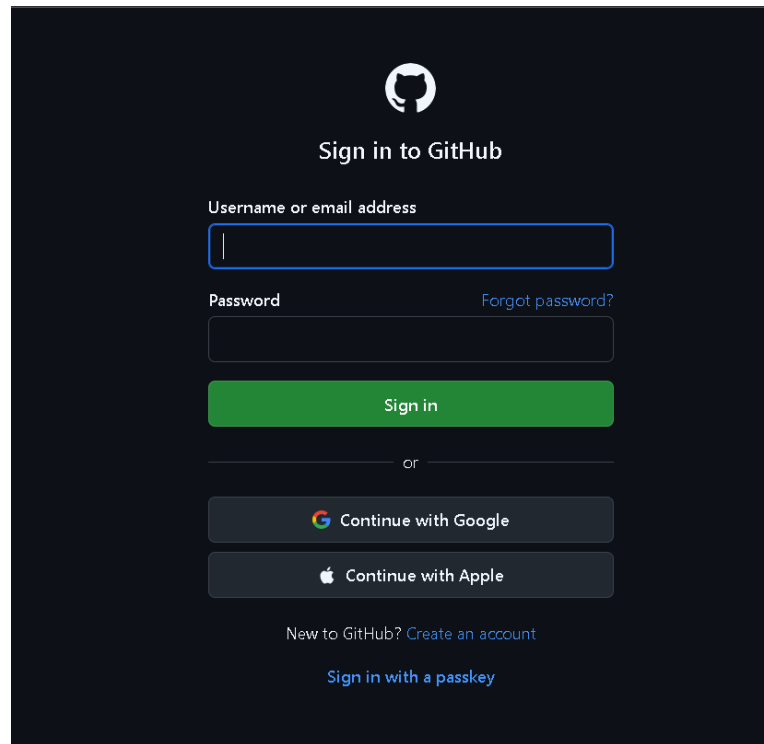
3.2.1. Registrasi Akun GitHub

- a. Buka browser dan akses situs GitHub.com



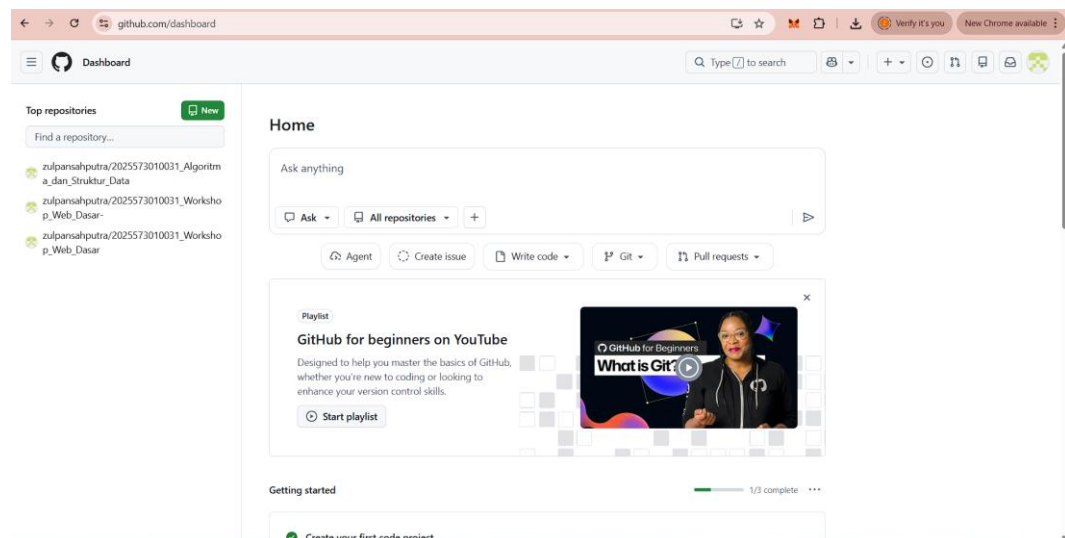
- b. Pilihlah menu Sign Up bila belum memiliki akun dan Sign in bila sudah memiliki akun.

- c. Pada tahap ini saya sudah memiliki akun GitHub dan saya memilih untuk sign in.



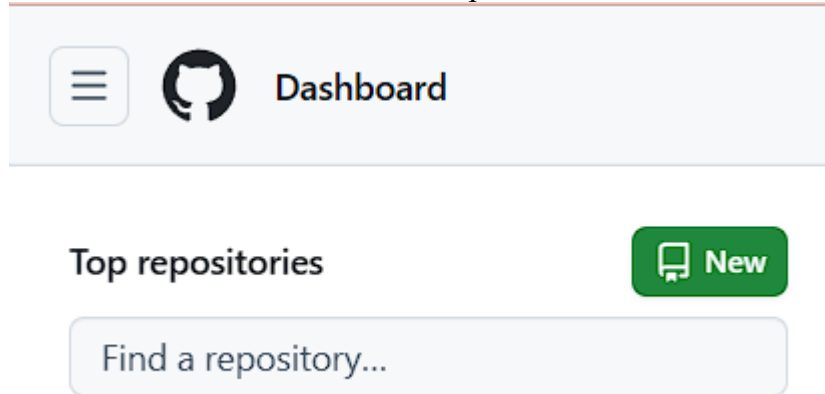
Masukkan email dan password akun yang dimiliki.

- d. Setelah proses register akun selesai akan masuk laman dashboard akun



3.2.2. Membuat *repository* baru pada GitHub

- a. Pada halaman dashboard akun terdapat sebuah tombol menu New



Tekan tombol tersebut untuk diarahkan pada menu pembuatan *repository* terbaru

- b. Pada halaman pembuatan *repository* diharuskan untuk mengisi dan memilih beberapa hal

The image shows the "Create a new repository" form on GitHub. It is divided into two sections: "1 General" and "2 Configuration". In the "General" section, there is a dropdown for "Owner" (currently showing "zulpansahputra"), a text field for "Repository name", and a text area for "Description" with a character count of "0 / 350 characters". In the "Configuration" section, there is a "Choose visibility" dropdown (set to "Public"), a toggle for "Add README" (currently "Off"), a dropdown for "Add .gitignore" (set to "No .gitignore"), and a dropdown for "Add license" (set to "No license"). A green "Create repository" button is at the bottom right.

- c. Isilah *repository* name untuk menyimpan nama dari *repository* terbaru
- d. Isilah description untuk menulis detail description project apakah yang akan dikerjakan pada *repository* berikut ini

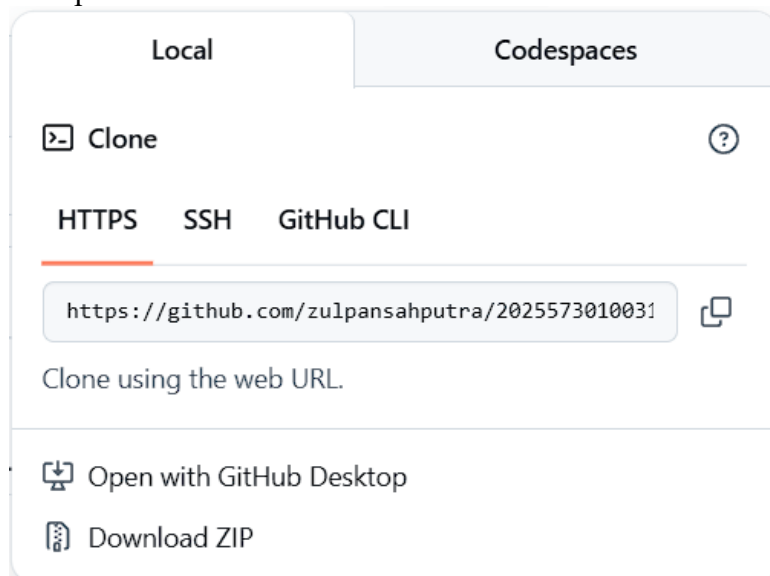
- e. Pada praktikum kali ini pilihan visibilitas menjadi public untuk dapat di akses oleh public melalui link *repository*
- f. Aktifkan pilihan add README untuk menambah sebuah file markdown di dalam *repository*
- g. Pada praktikum kali ini pilihan add .gitignore dan add license di langkahi saja tanpa perlu di pilih
- h. Klik tombol “Create Repository” untuk menyelesaikan proses pembuatan *repository*

3.2.3. Cloning repository GitHub ke local

- a. Setelah proses pembuatan *repository*, akan muncul halaman repository tersebut berada

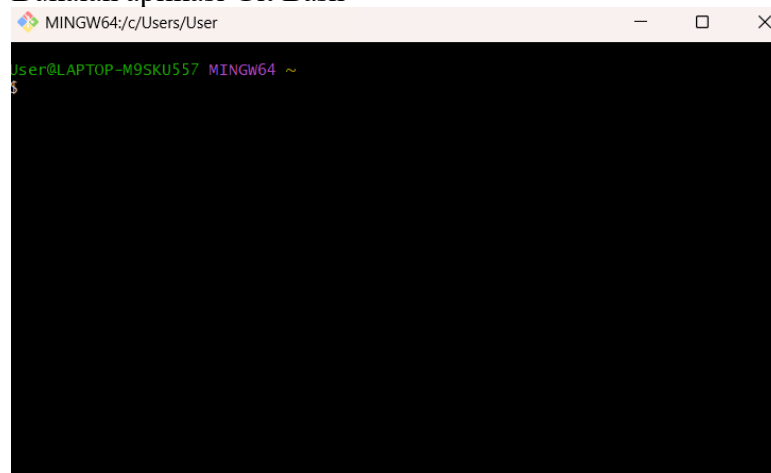


- b. Klik pada tombol code



- c. Salin link https tersebut

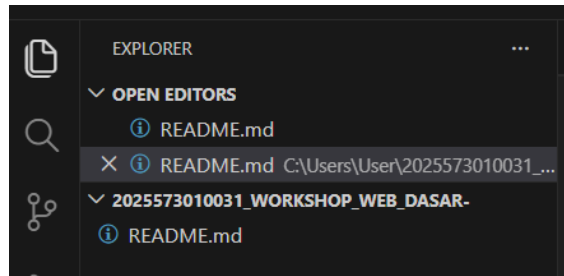
d. Bukalah aplikasi Git Bash



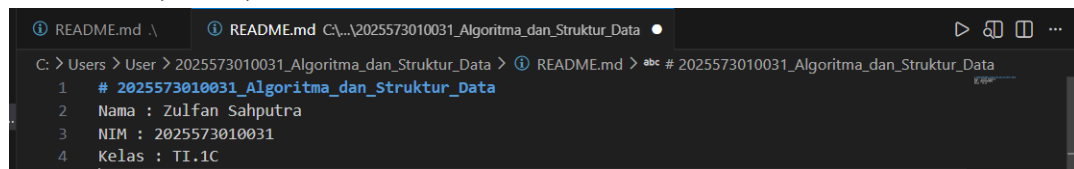
- e. Repository ini ingin saya simpan dalam direktori D:/
Jalankan sebuah command untuk memindahkan direktori terminal ke D:/
- f. Lakukan proses cloning repository pada directory tersebut dengan menjalankan sebuah command:
git clone "link https repository"
- g. Proses cloning *repository* dari GitHub server ke local device selesai

3.2.4. Editing README.md dan sinkronisasi *repository*

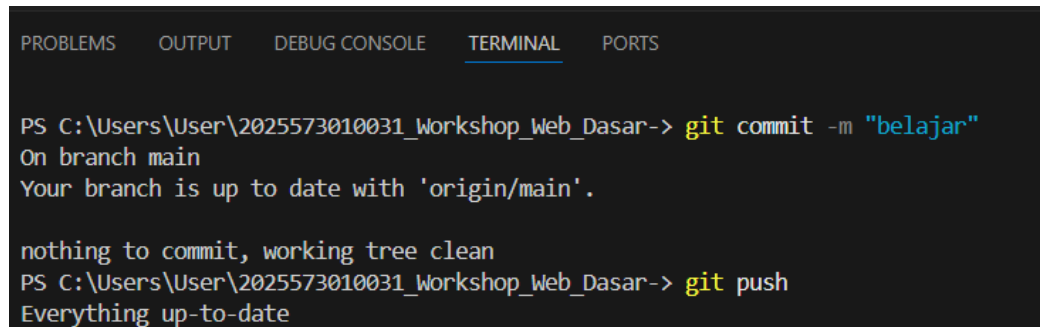
- a. Repository yang telah di-clone kemudian dibuka menggunakan Visual Studio Code



- b. File README.md diedit dengan menambahkan informasi berupa mata kuliah, nama, dan NIM



- c. Setelah pengeditan selesai, dilakukan proses *staging* menggunakan perintah `git add`
- d. Perubahan yang telah di-*staging* kemudian disimpan ke *repository* lokal menggunakan perintah `git commit`
- e. Selanjutnya dilakukan proses push untuk mengirim perubahan dari local *repository* ke remote *repository* pada GitHub



- f. Untuk memastikan sinkronisasi data antara *repository* lokal dan remote, dapat digunakan perintah `git pull`

BAB IV

HASIL DAN PEMBAHASAN

4.1. Hasil Praktikum

Hasil yang diperoleh:

- a. *Repository* berhasil dibuat
https://github.com/zulpansahputra/2025573010031_Workshop_Web_Dasar-
- b. README.md berhasil diperbarui
- c. Perubahan berhasil dipush ke GitHub

4.2. Pembahasan

Praktikum ini menunjukkan bahwa Git bekerja dengan sistem snapshot, bukan menyimpan perubahan per baris, tetapi menyimpan kondisi file pada waktu tertentu (*commit*).

Proses *staging* diperlukan agar Git mengetahui file mana yang akan disimpan. Tanpa perintah `git add`, file tidak akan masuk ke *commit*. Perintah `git push` mengirimkan commit dari local *repository* ke remote *repository* di GitHub, sedangkan `git pull` digunakan untuk mengambil perubahan terbaru dari server agar local *repository* tetap sinkron.

Dengan demikian, sinkronisasi dua arah dapat berjalan dengan baik selama prosedur dilakukan secara sistematis.

Selama praktikum, kemungkinan konflik dapat terjadi jika dua pengguna mengubah file yang sama. Git menyediakan mekanisme *merge* untuk menyelesaikan konflik tersebut.

BAB V

KESIMPULAN DAN SARAN

5.1. *Kesimpulan*

Penggunaan Git dan GitHub memungkinkan pengelolaan versi file secara terstruktur. Proses *clone*, *staging*, *commit*, *push*, dan *pull* terbukti efektif untuk menjaga sinkronisasi data antara local dan remote *repository*.

5.2. *Saran*

Mahasiswa disarankan untuk lebih memahami penggunaan branch dan merge sebagai pengembangan dari praktikum dasar ini.

DAFTAR PUSTAKA

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. Diakses dari <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>

GitHub Docs. (2024) GitHub Documentation. Diakses dari <https://docs.github.com>

OpenAI. (2026). *ChatGPT* [Large language model]. Diakses dari <https://chat.openai.com>